

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	D.HEMASREE	Reg. No.:	192524114
Section / Batch:	2025	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

CODE:

import sys

def bellman_ford(n, edges, start_node):

dist = [float('inf')] * n

dist[start_node] = 0

for _ in range(n - 1):

for u, v, w in edges:

```
    if dist[u] != float('inf') and dist[u] + w < dist[v]:  
        dist[v] = dist[u] + w
```

```
for u, v, w in edges:  
    if dist[u] != float('inf') and dist[u] + w < dist[v]:  
        return None
```

```
return dist
```

```
def floyd_warshall(n, edges):  
    dist = [[float('inf')] * n for _ in range(n)]  
  
    for i in range(n):  
        dist[i][i] = 0  
  
    for u, v, w in edges:  
        dist[u][v] = w  
  
    for k in range(n):  
        for i in range(n):  
            for j in range(n):  
                if dist[i][k] != float('inf') and dist[k][j] != float('inf'):  
                    if dist[i][j] > dist[i][k] + dist[k][j]:  
                        dist[i][j] = dist[i][k] + dist[k][j]  
  
    return dist
```

```
def main():  
    input_data = sys.stdin.read().split()  
  
    if not input_data:  
        return  
  
    idx = 0  
    n = int(input_data[idx])  
    m = int(input_data[idx + 1])  
    idx += 2  
  
    edges = []  
  
    for _ in range(m):  
        u = int(input_data[idx])
```

```

    v = int(input_data[idx + 1])
    w = int(input_data[idx + 2])
    edges.append((u, v, w))
    idx += 3

source = 0

print(f"--- Bellman-Ford (Source: {source}) ---")
bf_result = bellman_ford(n, edges, source)

if bf_result is None:
    print("Negative cycle detected!")
else:
    print(f"Distances: {bf_result}")

print("\n--- Floyd-Warshall (All-Pairs) ---")
fw_result = floyd_warshall(n, edges)

print("Distance Matrix:")
for row in fw_result:
    print(row)

if __name__ == "__main__":
    main()

```

OUTPUT:

QUESTIONS

Q1

Bellman-Ford vs Floyd's Algorithm (Comparison)

100 marks

1/1 answered

Q1 Bellman-Ford vs Floyd's Algorithm (Comparison)

100 marks

Given a graph, compute shortest paths using both algorithms. Compare single-source vs all-pairs approaches. Analyze time complexity differences. Evaluate memory usage. Discuss scalability. Generate insights on shortest path problems.

Your Code

```
1 import sys
2 def bellman_ford(n, edges, start_node):
3     dist = [float('inf')] * n
4     dist[start_node] = 0
5     for _ in range(n - 1):
6         for u, v, w in edges:
7             if dist[u] != float('inf') and dist[u] + w < dist[v]:
8                 dist[v] = dist[u] + w
9     for u, v, w in edges:
10        if dist[u] != float('inf') and dist[u] + w < dist[v]:
11            return None
12    return dist
13 def floyd_warshall(n, edges):
14     dist = [[float('inf')] * n for _ in range(n)]
15     for i in range(n):
16         dist[i][i] = 0
17     for u, v, w in edges:
18         dist[u][v] = w
19     for k in range(n):
20         for i in range(n):
21             for j in range(n):
22                 if dist[i][k] != float('inf') and dist[k][j] != float('inf') and dist[i][j] > dist[i][k] + dist[k][j]:
23                     dist[i][j] = dist[i][k] + dist[k][j]
24     return dist
25 def main():
26     input_data = sys.stdin.read().split()
27     if not input_data:
28         return
29     idx = 0
30     n = int(input_data[idx])
31     m = int(input_data[idx+1])
32     idx += 2
33     edges = []
34     for _ in range(m):
35         u = int(input_data[idx])
36         v = int(input_data[idx+1])
37         w = int(input_data[idx+2])
38         edges.append((u, v, w))
39         idx += 3
40     source = 0
41     print(f"--- Bellman-Ford (Source: {source}) ---")
42     bf_result = bellman_ford(n, edges, source)
43     if bf_result is None:
44         print("Negative cycle detected!")
45     else:
46         print(f"Distances: {bf_result}")
47     print("\n--- Floyd-Warshall (All-Pairs) ---")
48     fw_result = floyd_warshall(n, edges)
49     print("Distance Matrix:")
50     for row in fw_result:
51         print(row)
52 if __name__ == "__main__":
53     main()
```

Input

5 8
0 1 6
0 2 7
1 2 8
...

Output

Distance Matrix:
[0, 2, 7, 4, -2]
[inf, 0, 8, 5, -4]
[inf, -5, 0, -3, -9]
[inf, -2, 6, 0, -8]
[inf, inf, inf, inf, 0]

SECTION A – Comparative Analysis

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Comparative Analysis

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____